Jainil Shah
Batch: T23, Roll no: 2301106
Subject: ASD & D

Experiment 3: Deploying a Multi-Service Application Using Docker Compose

1. Aim

To design, containerize, and deploy a multi-service web application consisting of a Frontend, a Backend, and a Database using Docker Compose, and to study service orchestration, intercontainer networking, and volume-based data persistence.

2. Theory

Docker is a platform that allows applications to be packaged with all their dependencies into standardized units called containers. A real-world application rarely consists of a single component - it typically has a presentation layer (frontend), a business logic layer (backend/API), and a data layer (database). Running each of these as a separate container and managing them individually using docker run commands is tedious and error-prone.

Docker Compose solves this problem by allowing all services of an application to be defined declaratively in a single YAML file (docker-compose.yml). With one command, docker compose up, Compose builds images, creates a private network, starts all containers in the correct order, and connects them together.

Key concepts used in this experiment:

- Service: A container definition (built from a Dockerfile or a base image) representing one component of the application.
- Image vs Build: 'image' pulls a ready-made image (e.g. mysql:8) from Docker Hub; 'build' compiles a custom image from a local Dockerfile.
- Networking: Compose automatically creates a bridge network. Containers can reach each other using the service name as the hostname (Docker's internal DNS), instead of IP addresses.
- Port Mapping: 'host_port:container_port' exposes a container's port to the host machine so it can be accessed from a browser.
- Volumes: Persistent storage that exists outside the container's lifecycle, so data (like database files) survives container restarts and recreation.

- depends_on: Controls the startup order of services, though it does not wait for the application inside the container to be fully ready.

3. Software / Tools Required

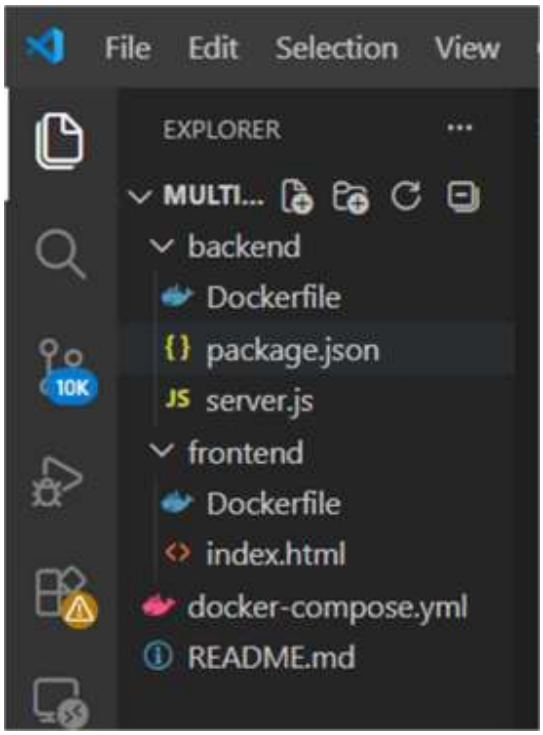
- Docker Engine (v20 or above)
- Docker Compose plugin (v2)
- A code editor (VS Code / nano)
- Web browser

4. System Architecture

The application follows a standard 3-tier architecture, with each tier running in its own container on a shared Docker bridge network defined in docker-compose.yml:

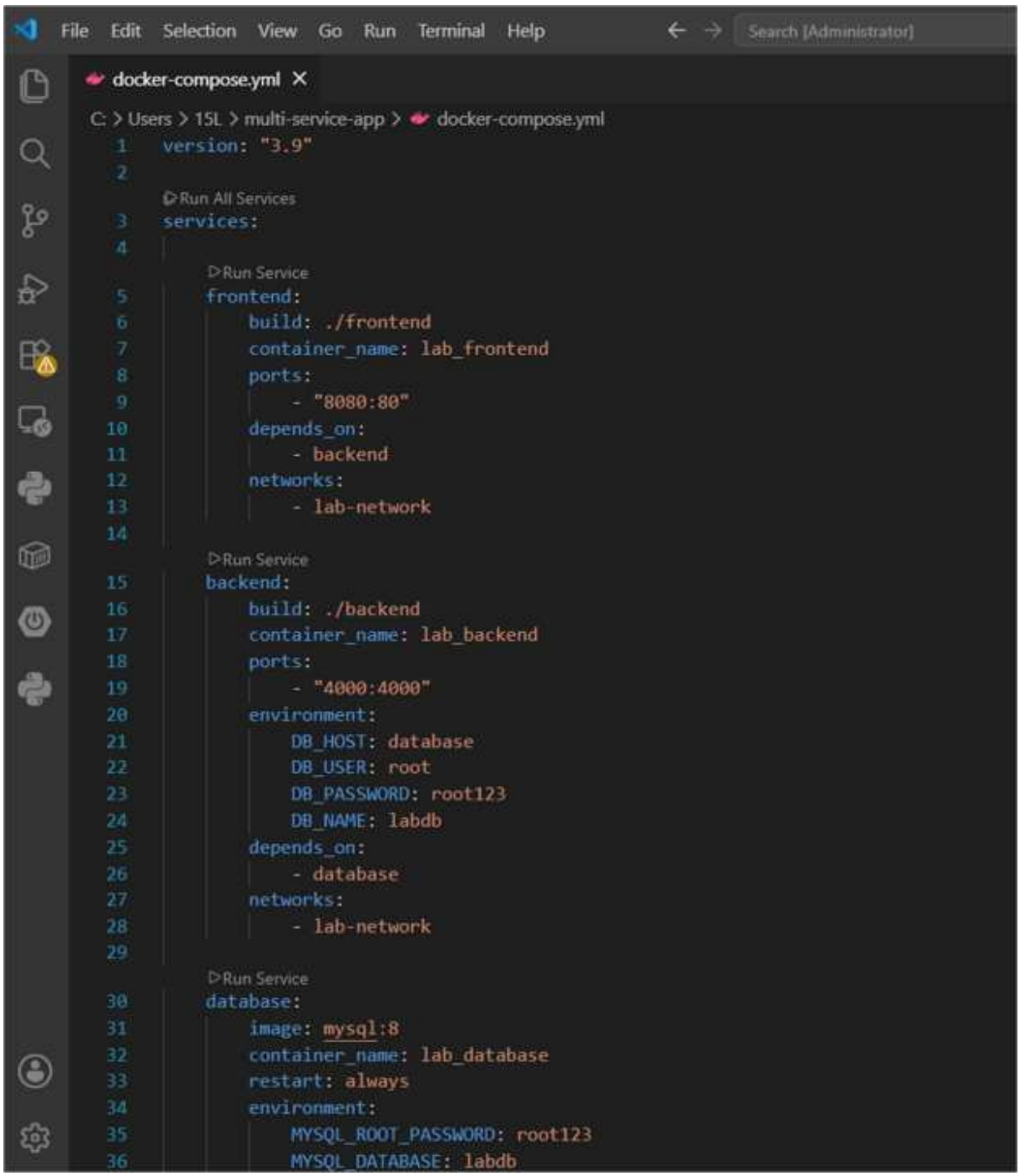
- Frontend: Nginx container serving a static HTML page (port 8080 on host).
- Backend: Node.js/Express REST API container that handles requests from the frontend and executes queries on the database (port 4000 on host).
- Database: MySQL 8 container that stores application data persistently using a named Docker volume (port 3306 on host).

5. Project Directory Structure



6. docker-compose.yml

This file defines all three services, their build context, exposed ports, environment variables, network, and the persistent volume for the database.



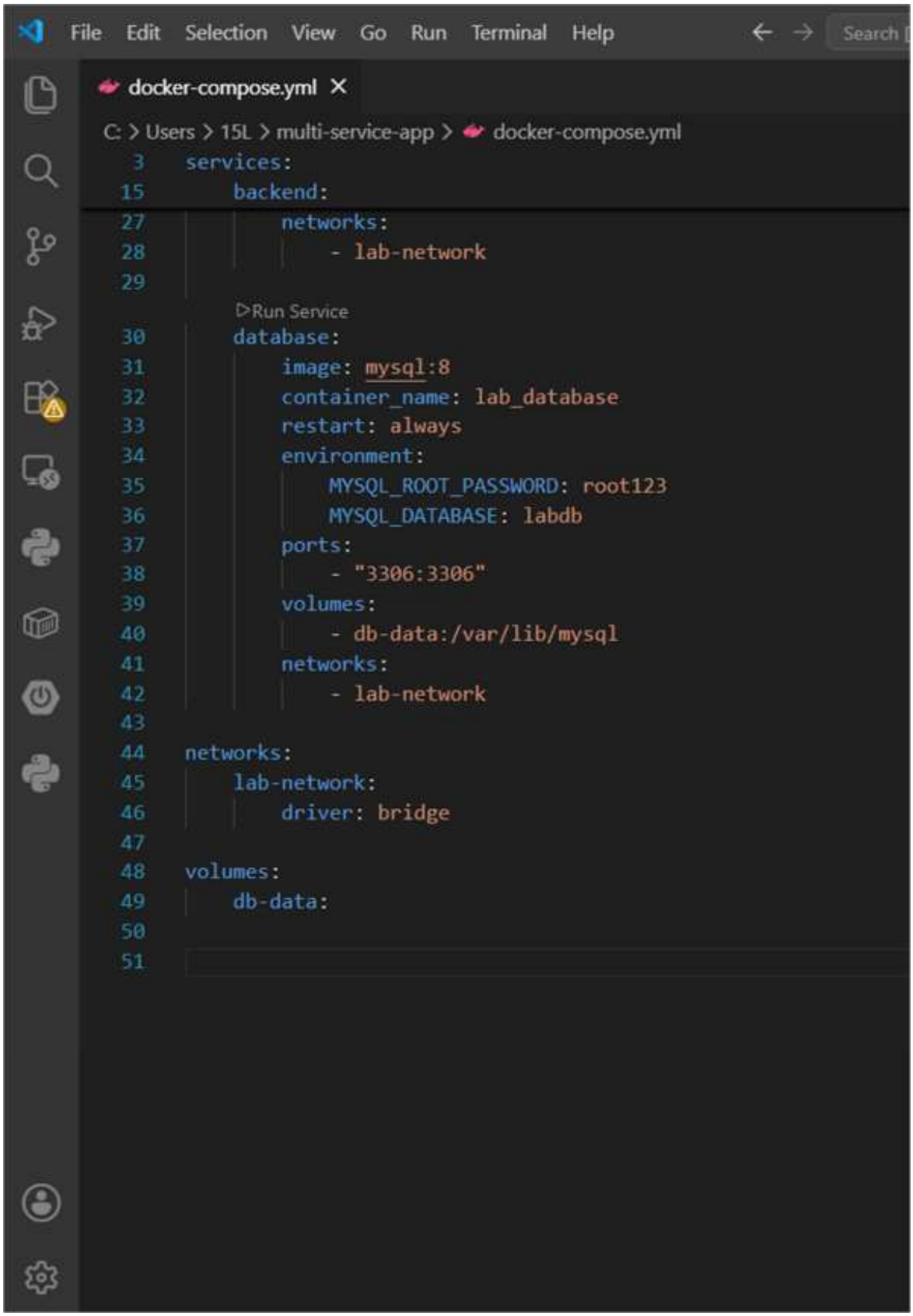

```yaml
version: "3.9"

services:

  frontend:
    build: ./frontend
    container_name: lab_frontend
    ports:
      - "8080:80"
    depends_on:
      - backend
    networks:
      - lab-network

  backend:
    build: ./backend
    container_name: lab_backend
    ports:
      - "4000:4000"
    environment:
      DB_HOST: database
      DB_USER: root
      DB_PASSWORD: root123
      DB_NAME: labdb
    depends_on:
      - database
    networks:
      - lab-network

  database:
    image: mysql:8
    container_name: lab_database
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: labdb
```


File Edit Selection View Go Run Terminal Help
Search
docker-compose.yml
C: > Users > 15L > multi-service-app > docker-compose.yml
Run Service
services:
backend:
networks:
- lab-network
database:
image: mysql:8
container_name: lab_database
restart: always
environment:
MYSQL_ROOT_PASSWORD: root123
MYSQL_DATABASE: labdb
ports:
- "3306:3306"
volumes:
- db-data:/var/lib/mysql
networks:
- lab-network
networks:
lab-network:
driver: bridge
volumes:
db-data:

7. Backend Code (server.js)

The backend is a Node.js/Express application that connects to the database container using the hostname 'database' (the Compose service name), not 'localhost'. This demonstrates Docker's internal DNS-based service discovery.



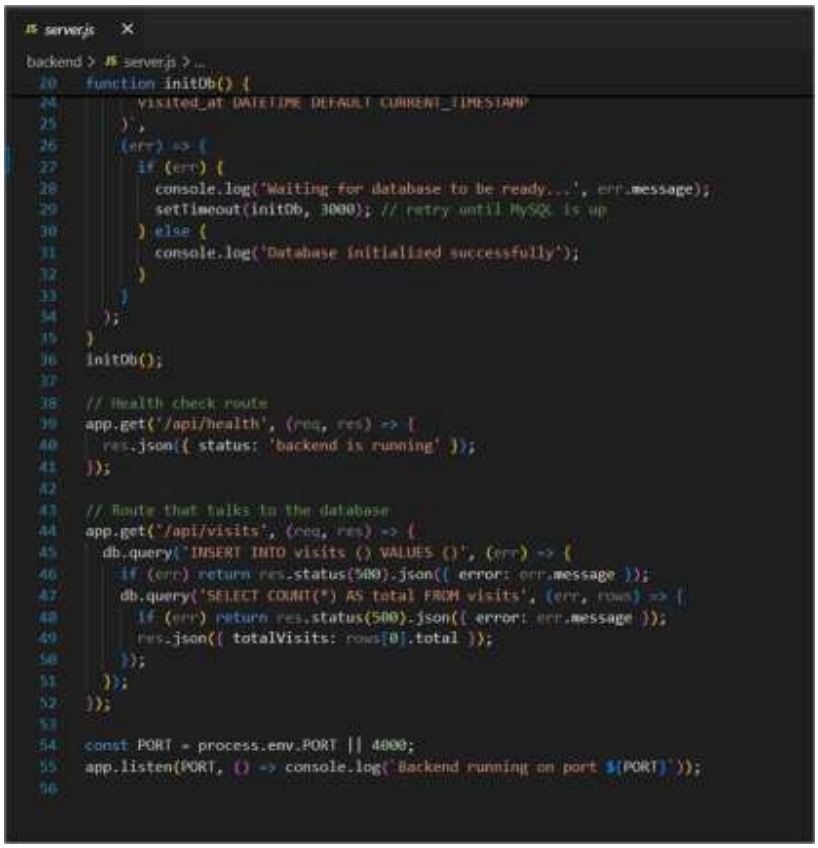
```
server.js
backend > server.js > ...
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// Connect to the "database" service using its Compose service name as hostname
const db = mysql.createPool({
  host: process.env.DB_HOST || 'database',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || 'root123',
  database: process.env.DB_NAME || 'labdb',
  waitForConnections: true,
  connectionLimit: 10
});

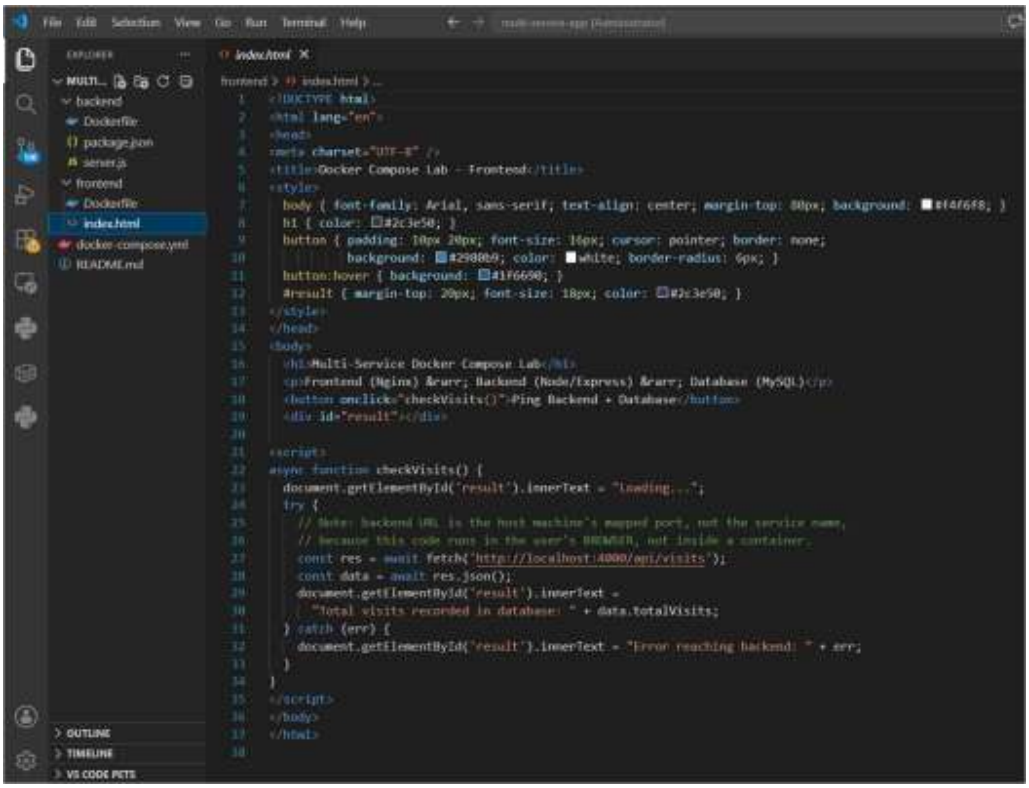
// Create table if it doesn't exist, then seed one row
function initDb() {
  db.query(
    `CREATE TABLE IF NOT EXISTS visits (
      id INT AUTO_INCREMENT PRIMARY KEY,
      visited_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )`,
    (err) => {
      if (err) {
        console.log('Waiting for database to be ready...', err.message);
        setTimeout(initDb, 3000); // retry until MySQL is up
      } else {
        console.log('Database initialized successfully');
      }
    }
  );
}
initDb();

// Health check route
app.get('/api/health', (req, res) => {
```



8. Frontend Code (index.html)

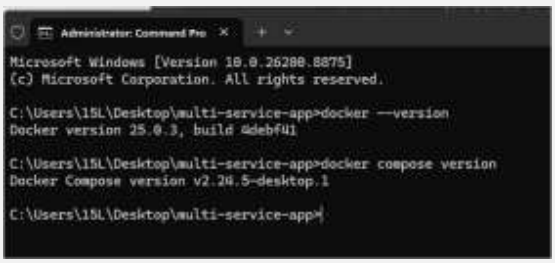
The frontend is a static HTML page served by Nginx. It calls the backend's REST API from the browser to fetch data recorded in the database.



9. Steps Performed

Step 1: Verify Docker installation

```
docker --version docker
compose version
```



Step 2: Create the project folder structure

```
mkdir multi-service-app && cd multi-service-app mkdir frontend
backend
```

Step 3: Write Dockerfiles and application code

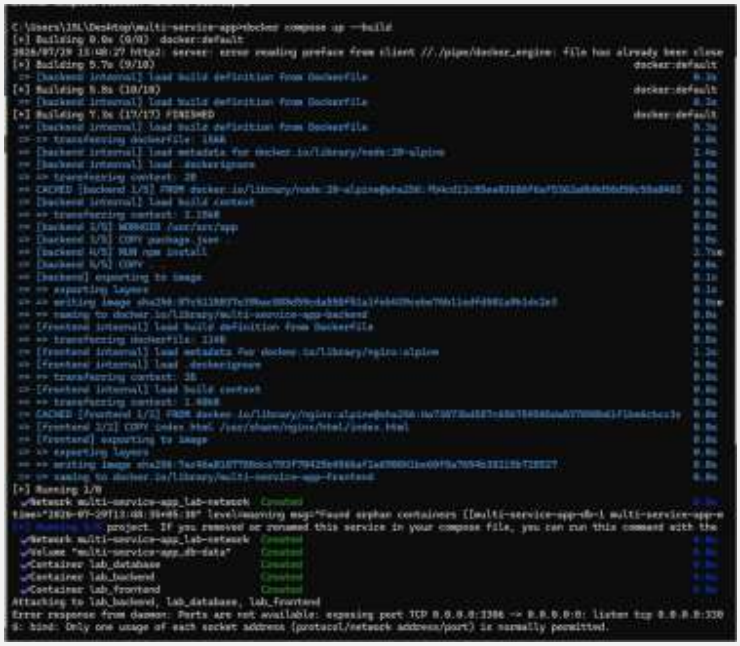
Created frontend/Dockerfile (Nginx base image), backend/Dockerfile (Node base image), and the application source files shown in Sections 7 and 8.

Step 4: Write docker-compose.yml

Defined the three services - frontend, backend, and database - along with a custom bridge network (lab-network) and a named volume (db-data) as shown in Section 6.

Step 5: Build and start all services

```
docker compose up –build
```



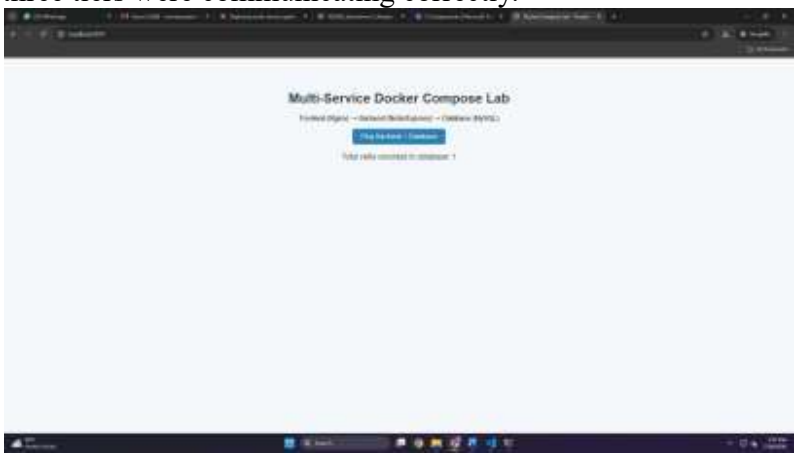
This command reads docker-compose.yml, builds the frontend and backend images from their Dockerfiles, pulls the mysql:8 image, creates the lab-network bridge network, creates the db-data volume, and starts all three containers.

Step 6: Verify all containers are running

```
docker compose ps
```

```
C:\Users\15L\Desktop\multi-service-app>docker compose ps
NAME           IMAGE                        COMMAND                  SERVICE    CREATED          STATUS          PORTS
lab_backend    multi-service-app-backend    "docker-entrypoint.s…"   backend    28 seconds ago   Up 27 seconds   0.0.0.0
:4000->4000/tcp
lab_database   mysql:8                      "docker-entrypoint.s…"   database   29 seconds ago   Up 27 seconds   33060/t
cp, 0.0.0.0:3307->3306/tcp
lab_frontend   multi-service-app-frontend   "/docker-entrypoint.…"   frontend   28 seconds ago   Up 27 seconds   0.0.0.0
:8081->80/tcp
```

Step 7: Test the application

- Opened http://localhost:8080 in a browser to view the frontend page.
- Clicked the 'Ping Backend + Database' button, which sent a request to http://localhost:4000/api/visits.
- The backend inserted a row into MySQL and returned the total visit count, confirming all three tiers were communicating correctly.
- 

- 

Step 8: Inspect logs and network

```
docker compose logs -f backend docker network
ls
docker network inspect multi-service-app_lab-network
```

Step 9: Stop and clean up

```
docker compose down        # stop and remove containers + network docker compose down -v     #
also remove the database volume
```

10. Observations

- All three containers (frontend, backend, database) started successfully with a single 'docker compose up' command.
- The backend was able to reach the database container using the service name 'database' as the hostname, confirming Docker Compose's built-in DNS-based service discovery.
- Data written to MySQL persisted across container restarts because it was stored in a named volume (db-data) rather than inside the container's writable layer.
- Running 'docker compose down -v' removed the volume and reset the visit counter to zero on the next run, confirming the role of volumes in data persistence.
- Port mapping allowed the host machine's browser to access services running inside isolated containers.

12. Docker Commands

COMMAND	DESCRIPTION
DOCKER COMPOSE UP --BUILD	Builds images and starts all 3 services (frontend, backend, database)
DOCKER COMPOSE UP -D --BUILD	Same, but runs in the background (detached mode)
DOCKER COMPOSE DOWN	Stops and removes containers + network
DOCKER COMPOSE DOWN -V	Also deletes the volume (wipes MySQL data — fresh start)
DOCKER COMPOSE DOWN --REMOVE-ORPHANS	Cleans up leftover containers from old runs (we needed this once)
DOCKER COMPOSE PS	Lists our containers and their status (Up/Down, ports)
DOCKER COMPOSE LOGS -F	Streams logs from all services live
DOCKER COMPOSE LOGS -F BACKEND	Streams logs from just the backend (useful for debugging DB connection)
DOCKER COMPOSE BUILD	Rebuilds images without starting containers
DOCKER COMPOSE STOP / DOCKER COMPOSE START	Stops/starts existing containers without deleting them
DOCKER COMPOSE EXEC BACKEND SH	Opens a shell inside the running backend container (for debugging)
DOCKER NETWORK LS	Lists Docker networks, including the one Compose created

DOCKER NETWORK INSPECT MULTI-SERVICE-APP_LABNETWORK	Shows which containers are on our custom network
DOCKER COMPOSE VERSION	Confirms Compose is installed and shows the version

11. Conclusion

A multi-service application with a Frontend, Backend, and Database was successfully containerized and deployed using Docker Compose. The experiment demonstrated how Compose simplifies multi-container orchestration through a single declarative YAML file, how containers communicate over a custom bridge network using service names, and how named volumes provide persistent storage independent of container lifecycles. This confirms Docker Compose as an effective tool for developing, testing, and deploying multi-tier applications in a consistent and reproducible manner.